House Price Prediction Using MLOps

1. Objective

To build a Linear Regression model for house price prediction and use MLflow, Joblib and FastAPI for basic MLOps implementation.

2. Problem Statement

A real estate company wants to predict the price of a house based on its size, number of bedrooms, number of bathrooms and age of the property.

For this practical, a Linear Regression model is trained and evaluated. MLflow is used to store experiment details, Joblib is used to save the trained model, and FastAPI is used for prediction through an API.

3. Dataset Preparation

The dataset file is **data/house_prices.csv**. It contains the following columns:

Column	Meaning
area_sqft	House area in square feet
bedrooms	Number of bedrooms
bathrooms	Number of bathrooms
house_age	Age of house in years
price	House price (target)

The five sample records given in the assignment are retained. Another 995 synthetic records are generated, giving **1000 records** in total.

```
price = (
    500000
    + 2800 * area_sqft
    + 250000 * bedrooms
    + 300000 * bathrooms
    - 40000 * house_age
    + noise
)
```

This formula is used only to generate sample data for the practical.

4. Data Preprocessing

The four input columns are selected as X and price is selected as y. The dataset is divided into 80% training and 20% testing.

```
X = df[FEATURES]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=10
)
```

Training records: 800 **Testing records:** 200

5. Model Training and Evaluation

```
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)
```

Metric	Result
MAE	INR 316,895.84
RMSE	INR 402,549.35
R2	0.9805

The R2 value obtained in the run is 0.9805. This result is for the synthetic dataset used in this exercise.

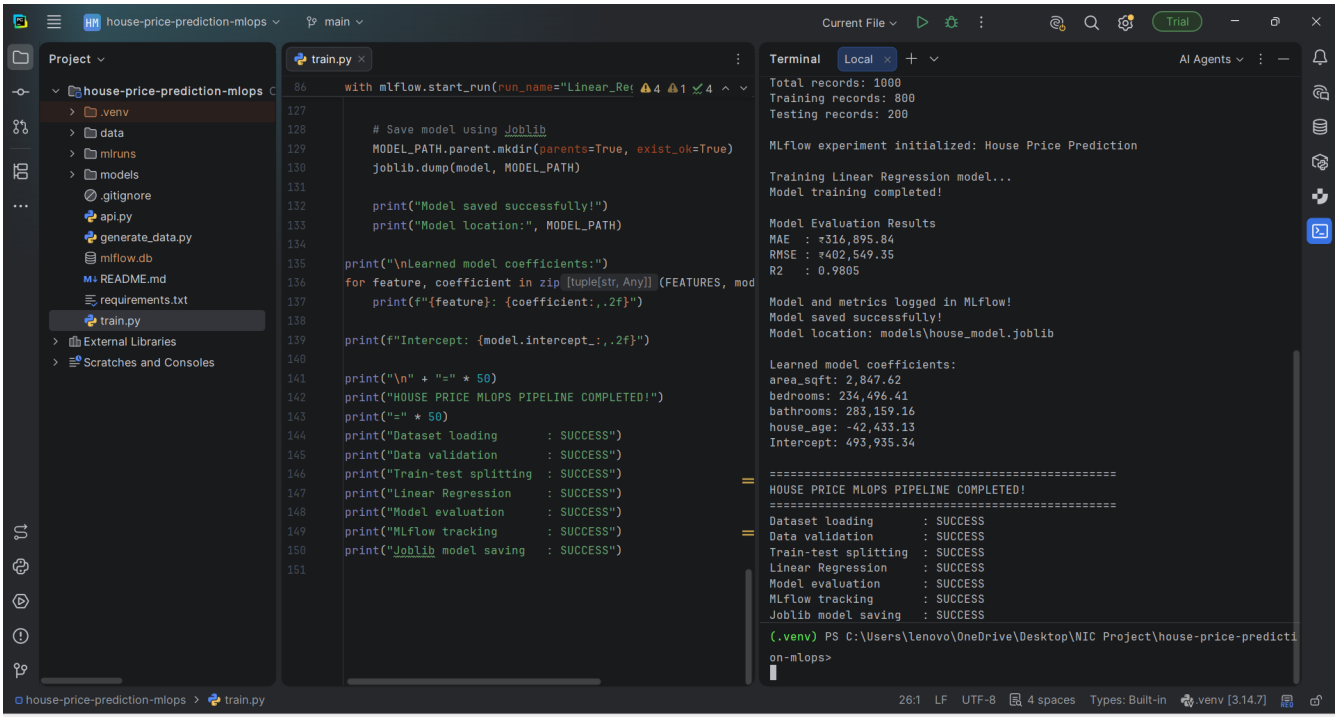


Figure 1 - Training output in PyCharm.

6. MLflow Tracking

The experiment name is **House Price Prediction** and the run name is **Linear_Regression_Run**. Parameters, metrics and the trained model are recorded.

```
mlflow.log_metric("mae", mae)
mlflow.log_metric("rmse", rmse)
mlflow.log_metric("r2", r2)

mlflow.sklearn.log_model(
    sk_model=model,
    name="house_price_model"
)
```

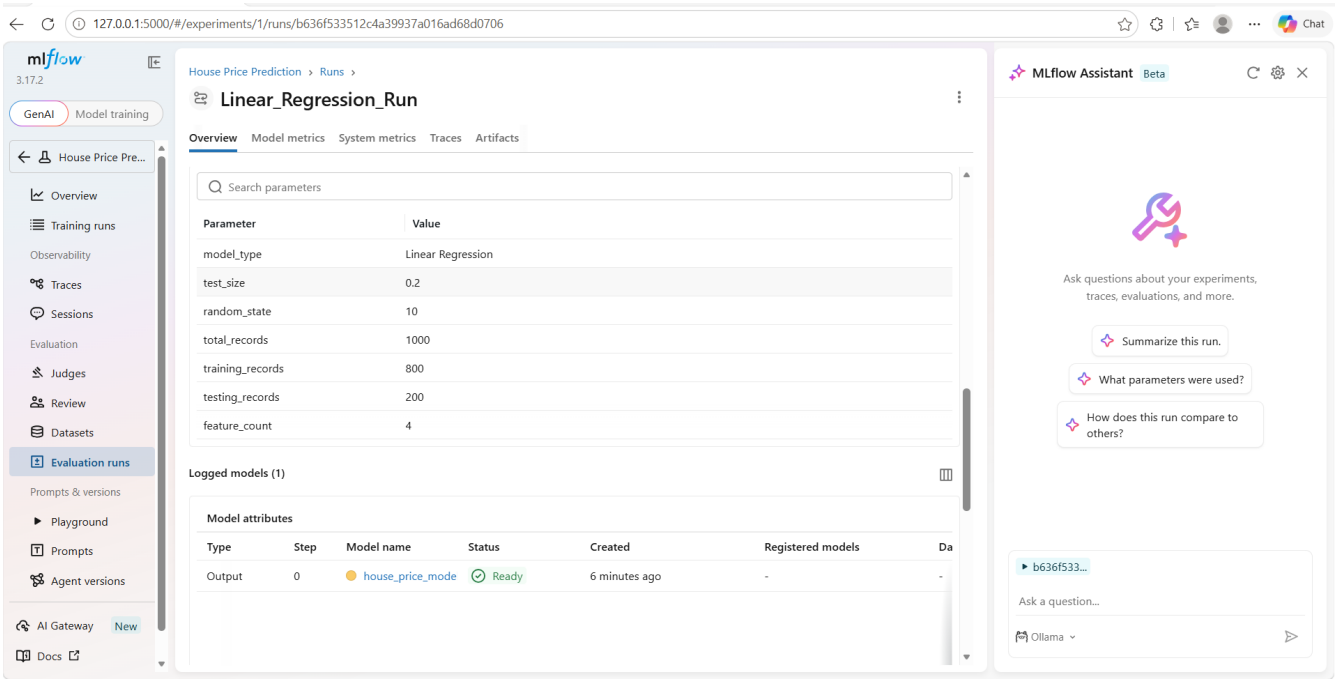


Figure 2 - MLflow run, parameters and model artifact.

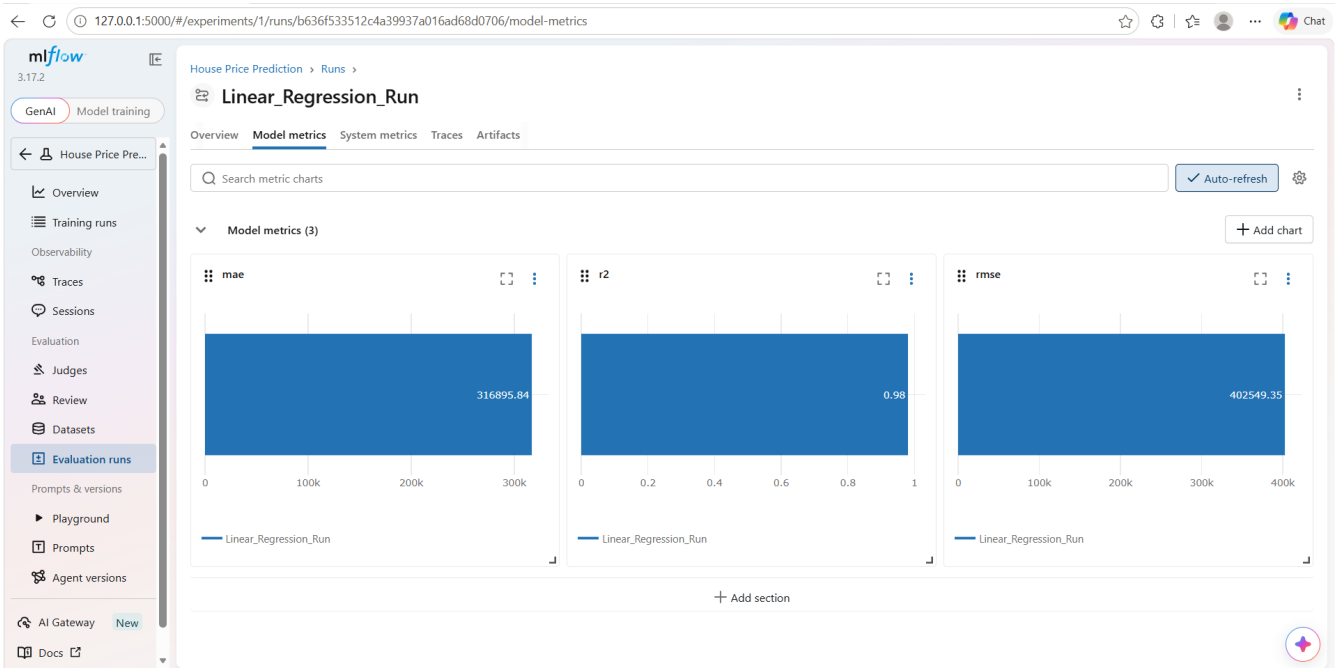


Figure 3 - MAE, R2 and RMSE recorded in MLflow.

7. Saving the Model

The trained model is saved as **models/house_model.joblib** using Joblib.

```
MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
joblib.dump(model, MODEL_PATH)
```

8. FastAPI Deployment

The saved model is loaded in api.py. POST /predict accepts the house details and returns the predicted price.

```
@app.post("/predict")
def predict(house: HouseInput):
    row = pd.DataFrame(
        [house.model_dump()],
        columns=FEATURES
    )
    prediction = float(model.predict(row)[0])
    return {
        "predicted_price": round(prediction, 2),
        "currency": "INR"
    }
```

9. Swagger Testing

The API is started with **uvicorn api:app --reload** and tested using Swagger UI.

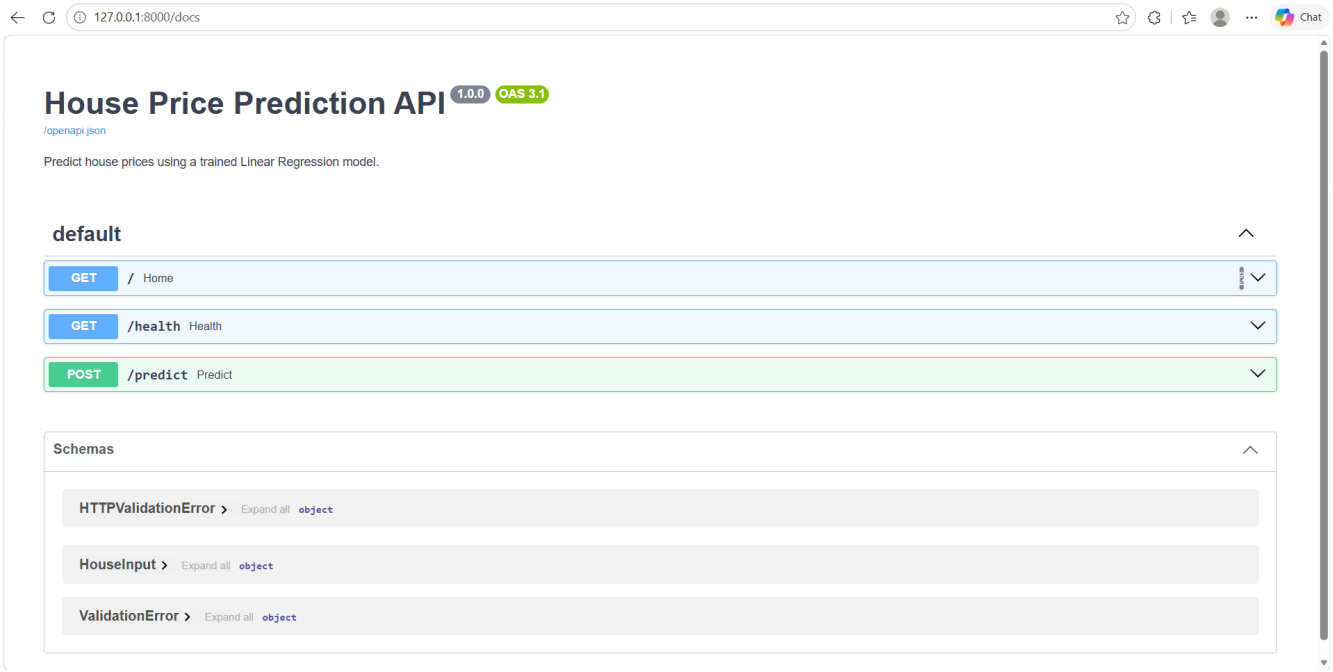


Figure 4 - FastAPI endpoints in Swagger UI.

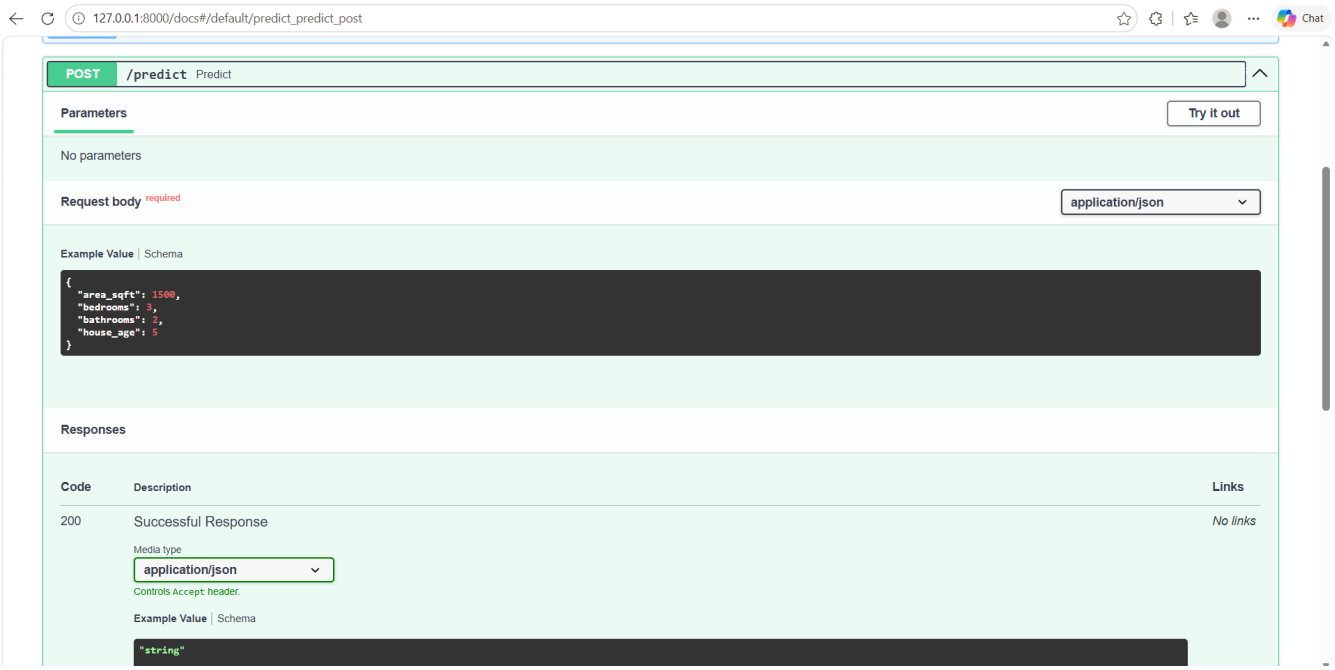


Figure 5 - Input given to POST /predict.

Test input: area_sqft = 1500, bedrooms = 3, bathrooms = 2 and house_age = 5.

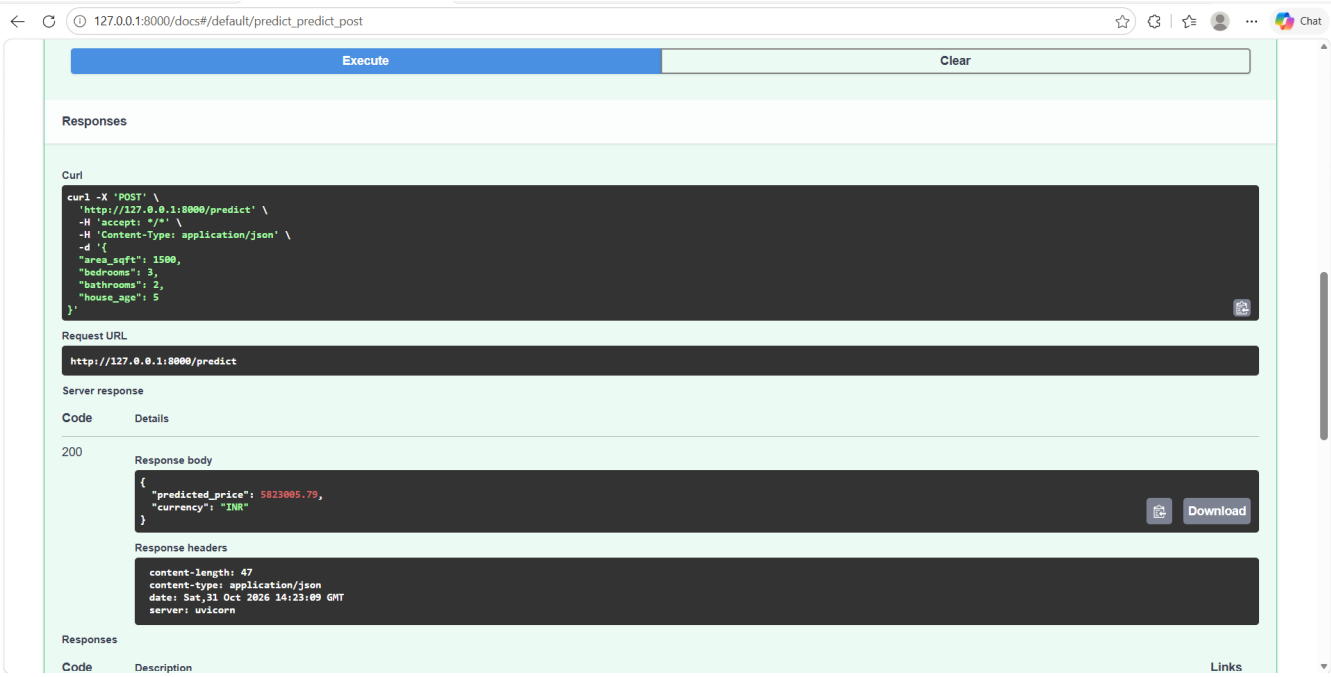


Figure 6 - Successful prediction response.

The request returned HTTP 200 and the predicted price was **INR 5,823,005.79**.

10. Project Files

File / Folder	Purpose
data/house_prices.csv	Dataset
models/house_model.joblib	Saved model
generate_data.py	Dataset generation
train.py	Training, evaluation and MLflow
api.py	FastAPI prediction API
requirements.txt	Python dependencies
.gitignore	Git ignore rules

11. Steps to Run

```
python -m pip install -r requirements.txt
python generate_data.py
python train.py
mlflow ui
uvicorn api:app --reload
```

12. Conclusion

The Linear Regression model was trained and evaluated successfully. The experiment was recorded in MLflow, the trained model was saved using Joblib, and the prediction API was tested successfully using FastAPI Swagger UI.

Codebase / GitHub Repository:

https://github.com/tejadande123-cyber/house-price-prediction-mlops